

OBJEKTORIENTERET ANALYSE, DESIGN OG IMPLEMENTATION

10: EXCEPTIONS AND ERROR HANDLING

Jan H. Mikkelsen
Cyber og Computerteknologi
2. semester – KBH – F26

AGENDA

- Introduction to exceptions
 - What causes exceptions
 - How did we handle special circumstances before exceptions?
 - Java build in exceptions
 - Exceptions as a class
 - Using exceptions
 - Throwing and catching exceptions
 - Create and use your own exceptions
 - The Finally block
 - The important stuff
 - Assignments
- 
- To understand the purpose of exceptions
 - To be able to write and use exceptions effectively
 - To be able to use the exception framework

RECAP - SOCKETS

2

- TrainServer

Includes only one single resource in this call ... `ServerSocket.close()` is auto-closed but everything else must be handled manually!!!

```
try ( ServerSocket serverSocket = new ServerSocket(PORT) ) {  
    serverSocket.setReuseAddress(true);  
    Socket clientSocket = serverSocket.accept();  
    System.out.println("Client connected: " + clientSocket.getInetAddress());  
    ...  
}
```

When closing a TCP connection, the OS sometimes hangs on (TIME_WAIT) to the port a bit. This line allows immediate reconnect. Without a quick dis- and reconnect would result in

This clientSocket, for instance, must be closed manually. The same applies for the in and out streams defined later in the code

Train Server starting on port 5001

Waiting for client...

Server error: Address already in use

RECAP - SOCKETS

3

- TrainClient

Opens up a TCP connection

Sets up a buffered reader to receive text from the server
raw bytes from socket → to characters → to buffered

```
try (  
    Socket socket = new Socket(host, port);  
    BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));  
    PrintWriter out = new PrintWriter(new OutputStreamWriter(socket.getOutputStream()), true);  
    BufferedReader userInput = new BufferedReader(new InputStreamReader(System.in)) ) {  
        System.out.println("Connected to server at " + socket.getInetAddress().getHostAddress() + ":" +  
            socket.getPort());  
        System.out.println();  
        System.out.println("Commands: list / search " + "/ add / delete / quit");  
        System.out.println();  
    }
```

Sets up a reader for keyboard input from the user

Sets up a writer to send text to the server. Adding the 'true' argument enables auto-flushing
PrintWriter already has internal buffering!!!

ERROR PRONE ACTIVITIES

- Consider a function, in C or similar, that does the following:

```
void readFile() {  
    // Open a file;  
    // Determine its size;  
    // Allocate that much memory;  
    // Read the file into memory;  
    // Close the file;  
}
```

- In this seemingly simple function call we have several sources for potential problems

ERROR PRONE ACTIVITIES

- Consider a function, in C or similar, that does the following:

```
void readFile() {  
    // Open a file;    ← Oops! The file does not exist!!  
    // Determine its size;    ← Oops! The file size cannot be determined!!  
    // Allocate that much memory;    ← Oops! Out of memory!!  
    // Read the file into memory;    ← Oops! File is corrupted!!  
    // Close the file;    ← Oops! Lost connection!!  
}
```

- So every single operation in this function call may result in some form of unexpected operation ... and this we need to handle

TRADITIONAL ERROR HANDLING (IN C)

- The traditional error-code approach to error handling is what we are often stuck with (here in C but it applies to most older languages)
- Depending on what has happened, we get some integer value back, as a representation of status
- No meaningful return information is provided
- The programmer must remember to check return values but nothing forces them to do so

```
errorCodeType readFile {  
    errorCodeType errorCode = 0;  
    // Variable declarations  
    FILE fp == fopen("MyFile.txt","r");  
    if (fp != NULL) {  
        fileLength = getFileLength();  
        if (fileLength > 0) {  
            buffer = malloc(fileLength*sizeof(int));  
            if (buffer != NULL) {  
                // allocation went OK, read  
                // the file into memory;  
                ...  
            }  
        }  
    }  
}
```

```
...  
if (readFailed()) {  
    errorCode = -1;  
}  
} else {  
    errorCode = -2;  
}  
} else {  
    errorCode = -3;  
    ...  
}  
return errorCode;  
}
```

TRADITIONAL ERROR HANDLING (IN C)

- More important (and worrying) is that the actual happy-path logic (reading the file) is buried several levels deep, while error handling is scattered throughout
- The actual code tends to drown in "if (...) { ... } else { // error handling }" constructs and the logical flow of the code is interrupted
- It is easy to lose the overview, and the error handling code may in fact itself introduce errors
- And again .. since we are not forced to do so, it is easy to forget to handle errors, causing buggy software

ERROR HANDLING IN JAVA

- In Java we operate with the Java exception hierarchy, where we have a clear distinction between 'Error' and 'Exception', both of which inherit from 'Throwable'
- Errors are catastrophic, JVM-level failures, e.g. something like `OutOfMemoryError`, `StackOverflowError`, or `VirtualMachineError`
 - If the JVM is out of memory, there is nothing sensible we can do about it
 - "The environment has catastrophically failed" → Error, do not bother catching
- Exceptions are the interesting part, as these represent conditions that are abnormal but recoverable, and this is where Java's design really shines compared to that C code you saw previously
 - "Something went wrong in my program's logic or environment" → Exception, let us try to handle it

EXCEPTIONS

- Exceptions are objects generated when the normal program flow is interrupted
- They are only generated when needed
- They can be used to *delegate information* of a failure or other events to a place in your program that is geared to handle it, thus improving the robustness of your software
 - This is dead central .. exceptions let you separate *where something goes wrong* from *where you deal with it*
- They do not save you any work, but they can improve the readability and logical flow of your code quite a lot
- Exceptions are all about organization and clarity ... not shortcuts

AGENDA

- Introduction to exceptions
 - What causes exceptions
 - How did we handle special circumstances before exceptions?
- Java build in exceptions
 - Exceptions as a class
- Using exceptions
 - Throwing and catching exceptions
- Create and use your own exceptions
- The Finally block
- The important stuff
- Assignments

CHECKED VERSUS UNCHECKED EXCEPTIONS

- **Checked exceptions** are conditions outside your control that any reasonable program should anticipate and recover from. The compiler enforces that you handle them
 - IOException — file might not exist
 - SQLException — database might be unavailable
 - FileNotFoundException — self-explanatory
- If you call a method that throws a checked exception, your code will not even compile unless you either catch it or declare that you will pass it up the call stack with throws
- **Unchecked exceptions** (subclasses of RuntimeException) represent programmer mistakes — bugs that should not happen if the code is correct
 - NullPointerException — you forgot to check for null
 - ArrayIndexOutOfBoundsException — your loop logic is wrong
 - IllegalArgumentException — someone passed invalid input
- The compiler does not force you to catch these, because theoretically you should write code that avoids them in the first place

CHECKED VERSUS UNCHECKED EXCEPTIONS

12

- Checked exceptions

```
//Will not compile without try/catch or throws declaration  
try {  
    FileReader file = new FileReader("test.txt");  
} catch (FileNotFoundException e) {  
    System.out.println("File not found: " + e.getMessage());  
}
```

Deal with it here!

```
public void readFile() throws FileNotFoundException {  
    FileReader file = new FileReader("test.txt");  
}
```

Let caller deal with the shit!

- Unchecked exceptions

```
int[] arr = new int[3];  
arr[10] = 5; // ArrayIndexOutOfBoundsException at runtime
```

BUILT-IN EXCEPTIONS

- Checked exceptions: the compiler ensures that these exceptions ARE handled in the code
- Unchecked exceptions: well, we still must live with stuff that cannot be solved at coding time
- Often caused by simple errors, but sometimes also complex errors
 - These are all programmer mistakes
- Useful to investigate the stack trace when debugging errors

List of unchecked runtime exceptions

1	ArithmeticException Arithmetic error, such as divide-by-zero.
2	ArrayIndexOutOfBoundsException Array index is out-of-bounds.
3	ArrayStoreException Assignment to an array element of an incompatible type.
4	ClassCastException Invalid cast.
5	IllegalArgumentException Illegal argument used to invoke a method.
6	IllegalMonitorStateException Illegal monitor operation, such as waiting on an unlocked thread.
7	IllegalStateException Environment or application is in incorrect state.
8	IllegalThreadStateException Requested operation not compatible with the current thread state.
9	IndexOutOfBoundsException Some type of index is out-of-bounds.
10	NegativeArraySizeException Array created with a negative size.
11	NullPointerException Invalid use of a null reference.
12	NumberFormatException Invalid conversion of a string to a numeric format.
13	SecurityException Attempt to violate security.
14	StringIndexOutOfBoundsException Attempt to index outside the bounds of a string.
15	UnsupportedOperationException An unsupported operation was encountered.

BUILT-IN EXCEPTIONS

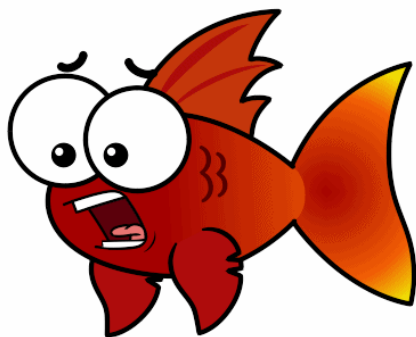
- It is worth noticing that in this list almost every single entry is entirely preventable with proper defensive coding, such as null checks, bounds checks, input validation
- This reinforces the point that unchecked exceptions represent bugs rather than recoverable conditions

List of unchecked runtime exceptions

1	ArithmeticException Arithmetic error, such as divide-by-zero.
2	ArrayIndexOutOfBoundsException Array index is out-of-bounds.
3	ArrayStoreException Assignment to an array element of an incompatible type.
4	ClassCastException Invalid cast.
5	IllegalArgumentException Illegal argument used to invoke a method.
6	IllegalMonitorStateException Illegal monitor operation, such as waiting on an unlocked thread.
7	IllegalStateException Environment or application is in incorrect state.
8	IllegalThreadStateException Requested operation not compatible with the current thread state.
9	IndexOutOfBoundsException Some type of index is out-of-bounds.
10	NegativeArraySizeException Array created with a negative size.
11	NullPointerException Invalid use of a null reference.
12	NumberFormatException Invalid conversion of a string to a numeric format.
13	SecurityException Attempt to violate security.
14	StringIndexOutOfBoundsException Attempt to index outside the bounds of a string.
15	UnsupportedOperationException An unsupported operation was encountered.

BREAK – PAUSE(FISK)

15



Jan H. Mikkelsen, OOADI10, CoCT, 2026

EXCEPTIONS

- The try block reads almost like plain English pseudo code. This reflects the happy-path
- All error handling is pushed to the right side column
- The order is important and the rule is to catch more specific exceptions before more general ones

```
void readFile() {  
    try {  
        // Open a file;  
        // Determine its size;  
        // Allocate that much  
        memory;  
        // Read the file into memory;  
        // Close the file;  
    }  
    ...  
}
```

```
catch (Exception e) { ... } // too broad - catches everything  
catch (FileOpenFailed e) { ... } // never reached!
```

```
catch(FileOpenFailed) {  
    // Error handling for file opening  
} catch(sizeDeterminationFailed) {  
    // Error handling for file size  
} catch(memoryAllocationFailed) {  
    // Error handling for malloc  
} catch(FileReadFailed) {  
    // Error handling for read failure  
} catch(FileCloseFailed) {  
    // Error handling for lost conn.  
    // etc.  
}  
}
```

EXCEPTIONS AS CLASSES

- A throwable is thrown by the Java statement *throw*
- A throwable contains a snapshot of the execution stack at the time it was created
- It usually has a message regarding the error that caused the exception
- It may also contain a cause
 - Another Throwable that can lead to a whole hierarchy of exceptions

java.lang.Throwable

java.lang.Exception

project.io

IOException

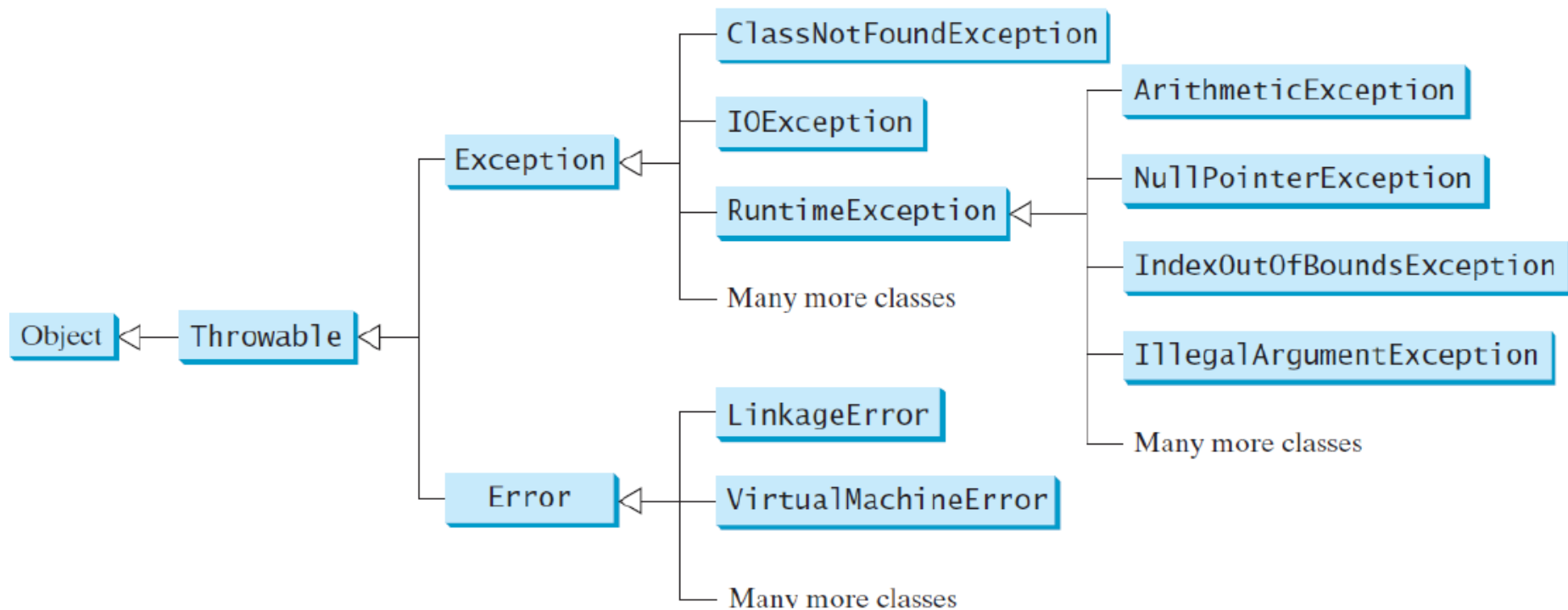
#msg: String
+IOException()
+IOException(String s)
+getMessage()

FileOpenException

#filename: String
+getFileName()
+getFile()

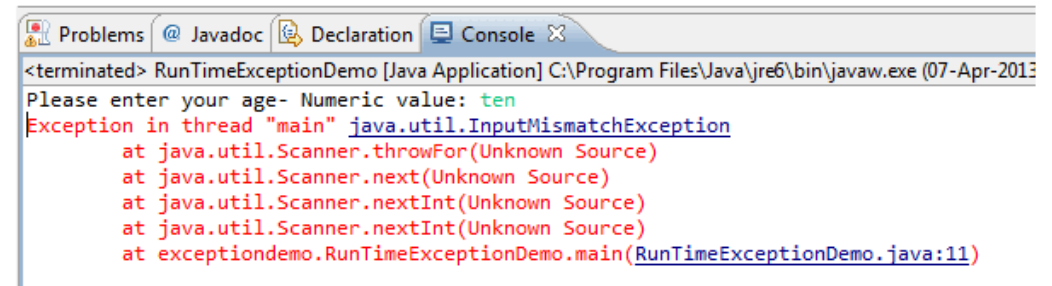
EXCEPTIONS – THE (ALMOST) FULL FAMILY TREE

18



EXCEPTIONS AS CLASSES

- String getMessage()
- Throwable getCause()
- String toString()
- Void printStackTrace()
- StackTraceElement[] getStackTrace()
- Throwable fillInStackTrace()
- Prints the full stack to stderr, which is most useful for debugging



```
<terminated> RunTimeExceptionDemo [Java Application] C:\Program Files\Java\jre6\bin\javaw.exe (07-Apr-2013)
Please enter your age- Numeric value: ten
Exception in thread "main" java.util.InputMismatchException
    at java.util.Scanner.throwFor(Unknown Source)
    at java.util.Scanner.next(Unknown Source)
    at java.util.Scanner.nextInt(Unknown Source)
    at java.util.Scanner.nextInt(Unknown Source)
    at exceptiondemo.RunTimeExceptionDemo.main(RunTimeExceptionDemo.java:11)
```

Sonatype Nexus
Nexus Repository Manager Servers

Nexus Repository Manager 3.x Server

Display Name:

Server ID:

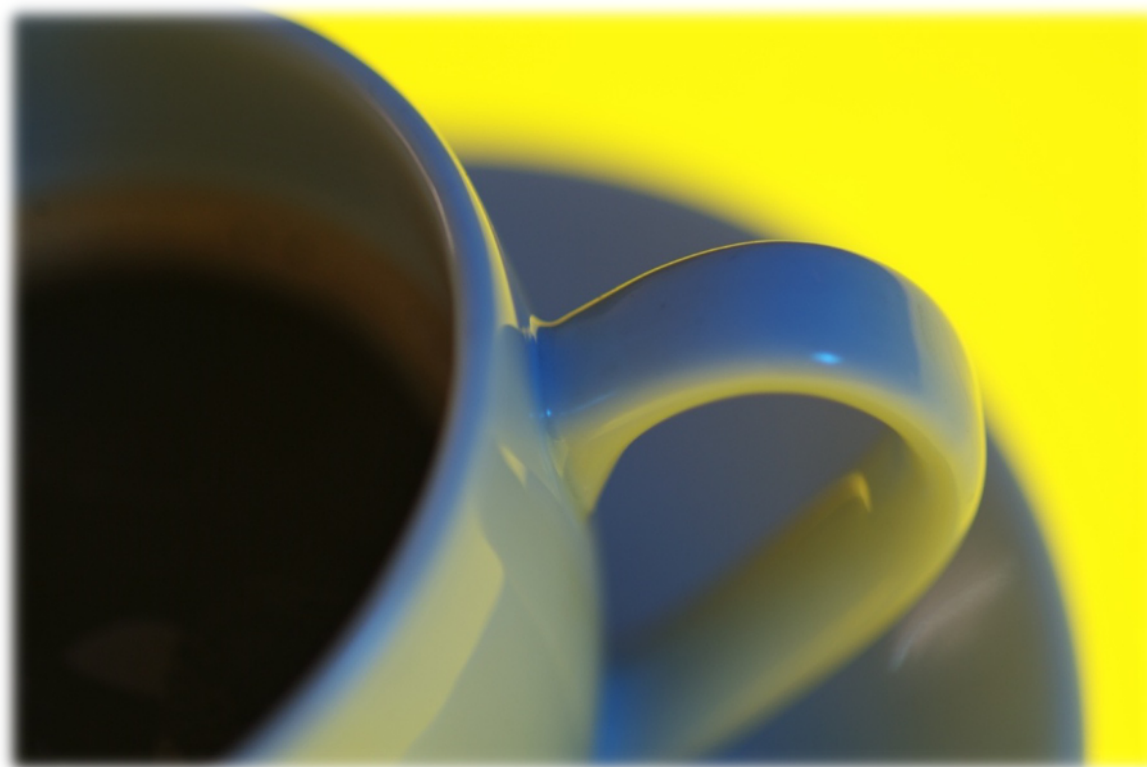
Server URL:

Credentials:

Nexus Repository Manager 3.x connection failed
javax.xml.bind.JAXBException: Implementation of JAXB-API has not been found on module path or classpath.
- with linked exception:
[java.lang.ClassNotFoundException: com.sun.xml.internal.bind.v2.ContextFactory]
 at javax.xml.bind.ContextFinder.newInstance(ContextFinder.java:278)
 at javax.xml.bind.ContextFinder.find(ContextFinder.java:421)
 at javax.xml.bind.JAXBContext.newInstance(JAXBContext.java:721)
 at javax.xml.bind.JAXBContext.newInstance(JAXBContext.java:662)
 at com.sonatype.nexus.api.zs.ex.a(SourceFile:33)
 at com.sonatype.nexus.api.zs.ex.b(SourceFile:24)
 at com.sonatype.nexus.api.zs.ez.handleResponse(SourceFile:54)
 at org.apache.http.impl.client.CloseableHttpClient.execute(SourceFile:223)
 at org.apache.http.impl.client.CloseableHttpClient.execute(SourceFile:165)
 at com.sonatype.nexus.api.zs.et.a(SourceFile:84)
 at com.sonatype.nexus.api.zs.es.getVersion(SourceFile:126)
 at com.sonatype.nexus.api.repository.v3.RepositoryManagerV3Client\$getVersion.call(Unknown Source)
 at org.sonatype.nexus.ci.config.Nxrm3Configuration\$DescriptorImpl.doVerifyCredentials(Nxrm3Configuration.groovy:92)
 at java.base/java.lang.invoke.MethodHandle.invokeWithArguments(MethodHandle.java:710)
 at org.kohsuke.stapler.Function\$MethodFunction.invoke(Function.java:396)
 at org.kohsuke.stapler.Function\$InstanceFunction.invoke(Function.java:488)

"BIKER BREAK!"

20



Jan H. Mikkelsen, OOAD110, CoCT, 2026

AGENDA

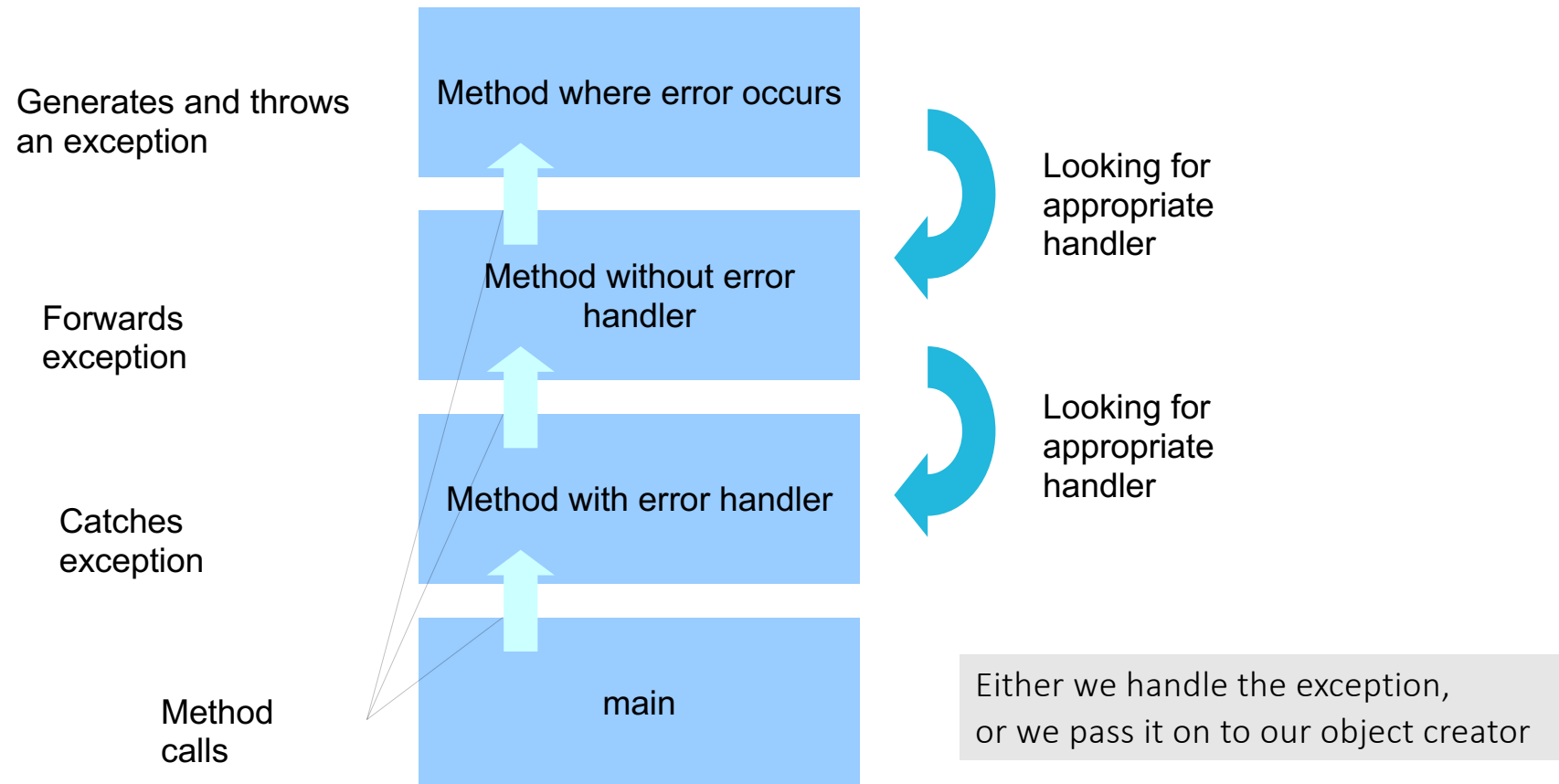
- Introduction to exceptions
 - What causes exceptions
 - How did we handle special circumstances before exceptions?
- Java build in exceptions
 - Exceptions as a class
- Using exceptions
 - Throwing and catching exceptions
- Create and use your own exceptions
- The Finally block
- The important stuff
- Assignments

USING EXCEPTIONS

- Exceptions are standard objects that must be a (possibly distant) subclass of `java.lang.Exception`
- When something goes wrong, an exception object is created (using a constructor, just like any other object) and then either handled immediately or thrown
- Once thrown, the program execution is halted
- The exception is then passed down the call stack until an appropriate code segment can process them. This is called catching an exception
- If an exception is not caught at some point, the program dies

THROWING EXCEPTIONS

23



THROWING EXCEPTIONS IN JAVA

- Where the exception occurs

```
public double computeOutput(double input) throws SingularMatrixException {  
    if (D.isSingular()) {  
        throw new SingularMatrixException("D is not invertible!");  
    }  
    // Computations  
    ...  
}
```

In the invoking method:

```
public Vector computeOutput(Vector inputs) throws  
    SingularMatrixException {  
    ...  
    for(Double input : inputs) {  
        v.add(computeOutput(input.doubleValue()));  
    }  
    ...  
}
```

CATCHING EXCEPTIONS IN JAVA

- Error handling code

```
public void compute() {  
    ...  
    try {  
        Vector v = system.computeOutput(theValues);  
        ...  
    } catch (SingularMatrixException sme) {  
        // Do the necessary to handle singular matrices  
        ...  
    }  
}
```

- Maybe we know in advance that the `system.computeOutput` may create an error if our matrix is singular – this is easy to check if we use Eclipse, as it will suggest us to catch a `SingularMatrixException`
- Then we can write to the user, that the matrix is singular, so we cannot compute (whatever it is we want to compute)

CATCHING A GENERIC EXCEPTION

- If we do not know what exception can happen, we can also catch the super

```
public void compute() {  
    ...  
    try {  
        Vector v = system.computeOutput(theValues);  
        ...  
    } catch (Exception e) {  
        // Do the necessary to handle some Exception  
        ...  
    }  
}
```



- Here we lose the possibility to handle specific errors
- And in consequence, we end up with lots of user responses that help very little

TRY-WITH-RESOURCES

- Faults can leave us hanging wrt. resources, e.g. files we attempt to open for some purpose
- If, for some reason we fail the try-part, then *finally*, we get to close the FileReader
- If we did not execute this code, 'fr' could potentially be hanging
 - Until the JVM is shut down from the host OS
 - Try to avoid that...
- Closing the file can itself throw an IOException, so you end up with a try-catch inside a finally, which is pretty ugly

```
import java.io.File;
import java.io.FileReader;
import java.io.IOException;

public class ReadData_Demo {

    public static void main(String args[]) {
        FileReader fr = null;
        try {
            File file = new File("file.txt");
            fr = new FileReader(file); char [] a = new char[50];
            fr.read(a); // reads the content to the array
            for(char c : a)
                System.out.print(c); // prints the characters one by one
        } catch (IOException e) {
            e.printStackTrace();
        } finally {
            try {
                fr.close();
            } catch (IOException ex) {
                ex.printStackTrace();
            }
        }
    }
}
```

AGENDA

28

- Introduction to exceptions
 - What causes exceptions
 - How did we handle special circumstances before exceptions?
- Java build in exceptions
 - Exceptions as a class
- Using exceptions
 - Throwing and catching exceptions
- Create and use your own exceptions
- The Finally block
- The important stuff
- Assignments

CREATING CUSTOM EXCEPTIONS

- Consider a bank offering an accounting system
- We know in advance, that someone may wish to withdraw too much money
- So, we model an error via an exception that we can cast anytime we wish to
- Notice, we can add any method as we wish according to an appropriate fault model

```
// File Name InsufficientFundsException.java
```

```
public class InsufficientFundsException extends Exception {  
    private double amount;  
  
    public InsufficientFundsException(double amount) {  
        this.amount = amount;  
    }  
  
    public double getAmount() {  
        return amount;  
    }  
}
```

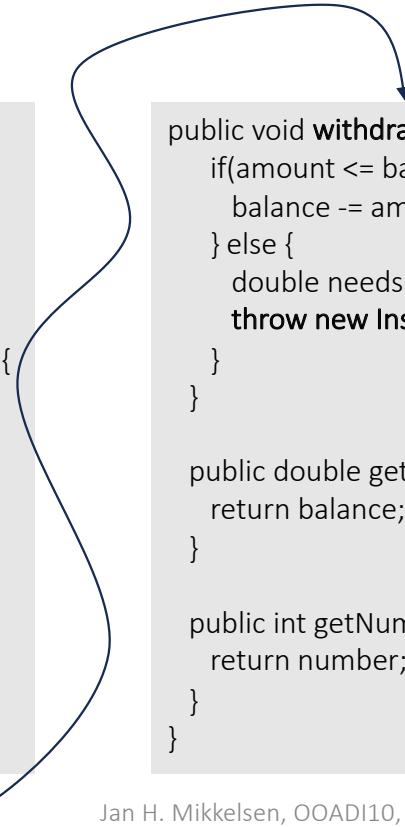
EXAMPLE – CHECKING ACCOUNT

- The bank model needs a bit more code – namely the bank account
- If withdrawal creates an error due to insufficient funds, we throw an appropriate exception

// File Name CheckingAccount.java

```
public class CheckingAccount {  
    private double balance;  
    private int number;  
  
    public CheckingAccount(int number) {  
        this.number = number;  
    }  
  
    public void deposit(double amount) {  
        balance += amount;  
    }  
  
    ....  
}
```

```
public void withdraw(double amount) throws InsufficientFundsException {  
    if(amount <= balance) {  
        balance -= amount;  
    } else {  
        double needs = amount - balance;  
        throw new InsufficientFundsException(needs);  
    }  
}  
  
public double getBalance() {  
    return balance;  
}  
  
public int getNumber() {  
    return number;  
}  
}
```



FINAL RUNTIME DEMO

- To test that it work, we need to setup our own simulation scenario
- We deposit first \$500, which obviously causes no concern (in our scenario)
- Then we withdraw – any call can be troublesome so we need to catch errors like our *InsufficientFundsException*
- We could also just catch any exceptions, but then we could not be certain that it was due to insufficient funds

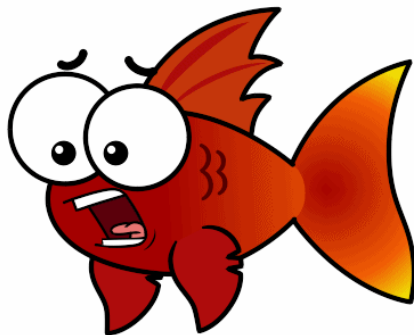
```
// File Name BankDemo.java
public class BankDemo {

    public static void main(String [] args) {
        CheckingAccount c = new CheckingAccount(101);
        System.out.println("Depositing $500...");
        c.deposit(500.00);

        try {
            System.out.println("\nWithdrawing $100...");
            c.withdraw(100.00);
            System.out.println("\nWithdrawing $600...");
            c.withdraw(600.00);
        } catch (InsufficientFundsException e) {
            System.out.println("Sorry, but you are short $" + e.getAmount());
            e.printStackTrace();
        }
    }
}
```


BREAK – PAUSE(FISK)

32



Jan H. Mikkelsen, OOAD10, CoCT, 2026

WHEN (AND WHEN NOT) TO USE EXCEPTIONS

33

- Some guidelines
- If your method encounters an abnormal condition that it can't handle, it should throw an exception
- Avoid using exceptions to indicate conditions that can reasonably be expected as part of the typical functioning of the method
- Exceptions indicate a broken contract
 - Design by Contract approach:
 - A method represents a contract between the client (the caller of the method) and the class that declares the method
 - Precondition
 - Postcondition

WHEN (AND WHEN NOT) TO USE EXCEPTIONS

34

//It is bad form to use exceptions as control flow

```
try {  
    int value = map.get(key); // throws if not found  
} catch (Exception e) {  
    // "handle" missing key  
}
```

//It is good for to handle expected conditions normally

```
if (map.containsKey(key)) {  
    int value = map.get(key);  
}
```

DIFFERENT END-OF-FILE APPROACHES

- This example uses `DataInputStream.readInt()`, which throws an `EOFException` when it runs out of data
- The infinite loop is intentionally broken by the exception
- It is considered bad practice to use exceptions for normal control flow rather than truly exceptional conditions
- It works, but it is using the exception system as a loop-break mechanism, which is exactly what exceptions are not meant for

```
import java.io.*;
class ExEOFa {
    public static void main(String[] args)
        throws IOException {
        if (args.length == 0) {
            System.out.println("Must give filename as first arg.");
            return;
        }
        FileInputStream fin;
        try {
            fin = new FileInputStream(args[0]);
        }
        catch (FileNotFoundException e) {
            System.out.println("Can't find file: " + args[0]);
            return;
        }
        DataInputStream din = new DataInputStream(fin);
        try {
            int i;
            for (;;) {
                i = din.readInt();
                System.out.println(i);
            }
        }
        catch (EOFException e) {
        }
        fin.close();
    }
}
```

Inf. loop

Silently "swallowed"

DIFFERENT END-OF-FILE APPROACHES

- Here, `InputStream.read()` returns `-1` (which is a sentinel value) when EOF is reached
- This is the cleaner approach, using a normal loop condition rather than exceptions
- This maps directly back to the checked vs. unchecked discussion
 - EOF is not an exceptional condition as it is the expected, normal end of reading a file
 - Using an exception to signal it (left) muddies the purpose of the exception system
 - The sentinel value approach keeps exceptions reserved for things that are genuinely abnormal
- This is therefore considered better style

```
import java.io.*;
class ExEOFb {
    public static void main(String[] args)
        throws IOException {
        if (args.length == 0) {
            System.out.println("Must give filename as first arg.");
            return;
        }
        FileInputStream in;
        try {
            in = new FileInputStream(args[0]);
        }
        catch (FileNotFoundException e) {
            System.out.println("Can't find file: " + args[0]);
            return;
        }
        int ch;
        while ((ch = in.read()) != -1) {
            System.out.print((char) ch);
        }
        System.out.println();
        in.close();
    }
}
```

AGENDA

37

- Introduction to exceptions
 - What causes exceptions
 - How did we handle special circumstances before exceptions?
- Java built-in exceptions
 - Exceptions as a class
- Using exceptions
 - Throwing and catching exceptions
- Create and use your own exceptions
- The Finally block
- The important stuff
- Assignments

THE FINALLY BLOCK

- Disregarding errors or not, sometimes some code must be executed
- The 'Finally' keyword is used to ensure that some code is always executed
- With no exceptions
 - Both try and finally block is executed
 - Catch is not executed
- Exception is detected and caught by catch
 - Rest of the statements in the try block are executed
 - Both the catch and finally blocks are executed
- Exception detected but not caught by catch
 - Rest of the statements in the try block are skipped
 - The finally block is executed

```
try {  
    statements;  
}  
catch (TheException ex) {  
    handling ex;  
}  
  
finally {  
    finalStatements;  
}
```

EXAMPLE USING FINALLY

- Being able to close a file or network, which has been created, is useful
- Why is it needed?
 - In cases where faults happens and we need to be sure code is executed
 - If we do not catch an exception, then the JVM stops execution – files may be hanging open, network sockets may be left dangling, etc.
- Why is it not used that often?
 - If we catch Exception as a generic fault, then we can always continue – however, it is hard to deal properly with a generic fault. Ever seen "An unknown error happened"??

```
class TestFinallyBlock {  
    public static void main(String args[]){  
        try{  
            //below code do not throw any exception  
            int data=25/5;  
            System.out.println(data);  
        }  
        //catch will not be executed  
        catch(NullPointerException e){  
            System.out.println(e);  
        }  
        //executed regardless of exception occurred or not  
        finally {  
            System.out.println("finally block is always executed");  
        }  
  
        System.out.println("rest of phe code...");  
    }  
}
```


EXAMPLE USING FINALLY

- The “modern” way of try-with-resources should be used when ever possible

```
FileReader fr = null;  
try {  
    fr = new FileReader("file.txt");  
} finally {  
    if (fr != null) fr.close();  
}
```

```
try (FileReader fr = new FileReader("file.txt")) {  
    // fr closed automatically  
}
```

EXAMPLE USING FINALLY

- Whenever a resource does not support auto-closable then we still need to do the finally part our selves

```
try {  
    db.setAutoCommit(false);  
    // ... do transaction  
    db.commit();  
} catch (SQLException e) {  
    db.rollback();  
} finally {  
    db.setAutoCommit(true); // always reset this, regardless  
}
```

```
lock.acquire();  
try {  
    // critical section  
} finally {  
    lock.release(); // must always release, no matter what  
}
```

EXAMPLE USING FINALLY

- How NOT to use finally
- Yes, we do close everything up nicely but we have no clue as to what caused an error
- This kind of generic catching is very problematic
- Only thing that is likely worse is the silently swallowing of exceptions as this tells us absolutely nothing

```
try {  
    // ... everything  
} catch (Exception e) {  
    System.out.println("An unknown error happened");  
} finally {  
    fin.close();  
    socket.close();  
}
```

```
catch (Exception e) {  
    // what do you actually DO here?  
    // - Is the data corrupted?  
    // - Did the network drop halfway through a transaction?  
    // - Was the file half-written? }
```

AGENDA

- Introduction to exceptions
 - What causes exceptions
 - How did we handle special circumstances before exceptions?
- Java build in exceptions
 - Exceptions as a class
- Using exceptions
 - Throwing and catching exceptions
- Create and use your own exceptions
- The Finally block
- The important stuff
- Assignments

THE IMPORTANT STUFF

- Exceptions are about separation of concerns. Exceptions let you separate where something goes wrong from where you deal with it
 - The try/catch structure enforces a clean architectural boundary
- Exceptions for exceptional conditions only and not for normal program flow
 - Misusing exceptions makes code harder to read and can hurt performance
- Checked vs unchecked is a design decision, not just syntax
 - Choosing the right category when designing your own exceptions reflects how you think about responsibility
- Custom exceptions make your code self-documenting
 - FileOpenFailed immediately reveal what went wrong. Exception tells you nothing. Investing in well-named, domain-specific exception classes adds value
- Catching order and granularity matter
 - Always catch specific before general. Each catch block should handle exactly what it claims to handle — nothing more

AGENDA

- Introduction to exceptions
 - What causes exceptions
 - How did we handle special circumstances before exceptions?
- Java build in exceptions
 - Exceptions as a class
- Using exceptions
 - Throwing and catching exceptions
- Create and use your own exceptions
- The Finally block
- The important stuff
- Assignments



ASSIGNMENTS

- 1) Consider the client code from previous course module exercise, what type of errors can you envision? Write at least three fault possibilities
- 2) Repeat with the server-side code that you may have. Write at least three fault possibilities
- 3) Some of the above faults and errors you are probably already catching. Consider by design, what to do when entering those fault conditions. Anything you can change from standard output to actual sensible behavior beside stopping some functionality?
- 4) Some faults you may not have covered by default. Define your own exceptions for those types of faults and address them in your code. Think about for example
 - a. A search for a specific train type most likely just lead to a null pointer exception, but that is not useful. Creating a TrainNotFoundException may be a more intuitive way to indicate and catch faults. But is it really a fault and if so, what type is it?
 - b. If your communication only allows, say 100 train descriptions to be send back in a single response, an error could also be that too many trains were found. How can you address this?